

Lesson 6: Next.js, File-Based Routing, and Multi-Page Apps

Mission Brief

Your React apps so far have lived on a single page. But a real portfolio site needs a Home page, an About page, a Projects listing, and individual project detail pages — with real, shareable URLs for each. Today you meet **Next.js**, a framework built on top of React that adds file-based routing, server-side rendering, and a production-ready project structure, and you use it to build a real multi-page site.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what Next.js adds on top of plain React
- Create pages using the App Router's file-based routing convention
- Use the `<Link>` component for client-side navigation between pages
- Create dynamic routes with bracket folders like `[slug]`
- Distinguish Server Components from Client Components at a beginner level

What You'll Build

A small Next.js site with a Home page, an About page, a Projects listing page, and a dynamic `/projects/[slug]` page that shows different content depending on the URL.

Prerequisites

- Comfortable with components, props, and state (Lessons 1-3)
- A Vite-based React project is not required for this lesson — Next.js projects are structured differently

Key Concepts

- What a framework adds beyond a UI library (routing, bundling, conventions)
- The `app/` directory and file-based routing
- `page.js` (or `.jsx`) as the file that defines a route's content
- The `<Link>` component vs. a plain `<a>` tag
- Dynamic segments: `app/projects/[slug]/page.js`
- Server Components (default) vs. Client Components (`"use client"`)

Concept Explanation

Plain React (as set up by Vite) gives you components and a way to render them — but it doesn't decide how URLs map to content, how your project should be organized, or how the server and browser split the work. **Next.js** is a framework that answers all of that for you with sensible defaults, so teams can build production sites faster and more consistently.

The biggest thing Next.js's **App Router** adds is **file-based routing**: instead of writing routing configuration by hand, the folder structure of your `app/` directory is your site's URL structure.

```
src/app/
  page.js           → the route "/"
  about/
    page.js         → the route "/about"
  projects/
    page.js         → the route "/projects"
    [slug]/
      page.js       → the route "/projects/anything-here"
```

Each `page.js` file exports a React component (a Server Component, by default) whose return value becomes that route's content:

```
// src/app/about/page.js
export default function AboutPage() {
  return (
    <main>
      <h1>About This Mission</h1>
      <p>I'm a developer who builds space-themed learning tools.</p>
    </main>
  );
}
```

To navigate between pages without a full browser reload (keeping the app feeling instant), Next.js provides a `<Link>` component instead of a plain `<a>` tag:

```
import Link from "next/link";

function NavBar() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/about">About</Link>
      <Link href="/projects">Projects</Link>
    </nav>
  );
}
```

A plain `<a href="/about">` would technically work, but it triggers a full page reload from the server every time; `<Link>` intercepts the click and swaps content on the client, preserving app state and loading only what's needed.

For pages whose content depends on a variable part of the URL — like a specific project's detail page — Next.js uses **dynamic segments**, written as a folder name in square brackets: `app/projects/[slug]/page.js`.

Whatever the visitor typed in that URL position becomes available to your component through its `params` prop:

```
// src/app/projects/[slug]/page.js
import { projects } from "@/data/projects";

export default function ProjectDetailPage({ params }) {
  const project = projects.find((p) => p.slug === params.slug);

  if (!project) {
    return <p>Project not found.</p>;
  }

  return (
    <main>
      <h1>{project.title}</h1>
      <p>{project.description}</p>
    </main>
  );
}
```

Visiting `/projects/ecommerce-page` sets `params.slug` to `"ecommerce-page"`, and the component looks up the matching project from your data file. This is exactly how one single file can serve unlimited "pages" — one per project — without writing a separate file for each.

Finally, Next.js's App Router introduces a distinction between **Server Components** (the default — they render on the server, can't use hooks like `useState`, and ship less JavaScript to the browser) and **Client Components** (opted into with a `"use client"` directive at the very top of the file — these can use state, effects, and event handlers, exactly like the React you've already learned). A component needs `"use client"` the moment it uses `useState`, `useEffect`, `onClick`, or any other browser-interactive feature:

```
"use client";
import { useState } from "react";

export default function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? "❤️" : "💔"}</button>;
}
```

Without `"use client"` at the top, using `useState` in an App Router file throws an error, because Server Components render in an environment with no browser event loop or interactivity — the directive is what tells Next.js "ship this one to the browser and let it be interactive."

Guided Walkthrough

1. In your Next.js project (already scaffolded for Project 09), open `src/app/page.js` and confirm it renders as your site's home page at `/`.
2. Create `src/app/about/page.js` exporting a simple component, run the dev server (`npm run dev`), and visit `/about` directly in the browser to confirm the route works.
3. Create a `NavBar` component using `<Link>` for Home/About/Projects, and render it inside your root `layout.js` so it appears on every page.
4. Build `src/app/projects/page.js` that imports a `projects` data array and renders a `<Link>` to `/projects/${project.slug}` for each one.
5. Build the dynamic `src/app/projects/[slug]/page.js` shown above, and confirm clicking each project link on the listing page navigates to a detail page with that project's specific content — with no full-page reload (watch the Network tab; you should see no full document reload, just data).

Common Bugs & Debugging Tips

- Using `useState` (or any hook) in a file without `"use client"` at the very top throws an error like "You're importing a component that needs useState... This React hook only works in a client component" — add the directive as the literal first line of the file.
- Forgetting `export default` on a `page.js` file means Next.js can't find the route's content — every `page.js` must default-export a component.
- A typo in a bracket folder name (`[slugs]` instead of `[slug]`) means `params.slug` is `undefined` in your component — the folder name *is* the prop key.
- Using a plain `<a href="/about">` works but causes a visible full-page flash on navigation — switch to `<Link href="/about">` for smooth client-side transitions.

Check Your Understanding

1. What determines a Next.js App Router route's URL — configuration code, or folder structure?

2. What does wrapping a folder name in square brackets, like `[slug]`, accomplish?
3. Why use `<Link>` instead of a plain `<a>` tag for internal navigation?
4. What does the `"use client"` directive do, and when do you need it?
5. Where does a dynamic route's URL segment show up inside your page component?

Mini Quiz + Answer Key

- Q1.** In the Next.js App Router, what determines a page's URL? A) A routing config file B) The folder structure inside `app/` C) The component's function name D) The CSS class name
- Q2.** What does `app/projects/[slug]/page.js` let you do? A) Create exactly one static page B) Serve a different page per URL value, read via `params.slug` C) Redirect to another site D) Nothing without extra config
- Q3.** Why prefer `<Link href="/about">` over `<a href="/about">` for internal navigation? A) `<a>` doesn't work at all in Next.js B) `<Link>` avoids a full page reload and feels instant C) `<a>` is deprecated in HTML D) There's no difference
- Q4.** When must a component file start with `"use client"`? A) Always B) When it uses hooks like `useState` / `useEffect` or event handlers C) Only for the home page D) Never, it's optional

Answer Key: 1-B, 2-B, 3-B, 4-B

Lesson Recap

You learned that Next.js adds file-based routing, `<Link>` for client-side navigation, dynamic `[slug]` segments for data-driven pages, and the Server/Client Component split via `"use client"` — the exact structure powering Project 09's multi-page portfolio site.

Homework

Add a `/contact` page with a controlled form (from Lesson 5's pattern, remembering `"use client"` since it uses `useState`), and add it to the shared `NavBar`. Then add a fourth project to your data file and confirm a new detail page automatically works at `/projects/your-new-slug` with zero new route files.

Portfolio Project Connection

Project 09's entire site structure — Home, About, Projects listing, and per-project detail pages — is built directly from this lesson's file-based routing, `<Link>` navigation, and dynamic segments.